

CHAPITRE 2

Fondements Théoriques et Conception du Système

1 .Fondements Théoriques

- 1.1 IA, ML et Deep Learning
- 1.2 Vision par ordinateur et Réseaux de Neurones
- 1.3 détection d'objets
- 1.4 Reconnaissance optique de caractères (OCR)

2.choix technologiques

→ YOLO11m et PP-OCrv5 justifiés ici

3.Spécification des besoins

- 3.1 Besoins fonctionnels
- 3.2 Besoins non fonctionnels

4.Architecture globale de la solution

- 4.1 Architecture logique (Frontend / Backend / IA / DB)
- 4.2 Architecture physique (on-premise, flux de données)

5.Conception UML

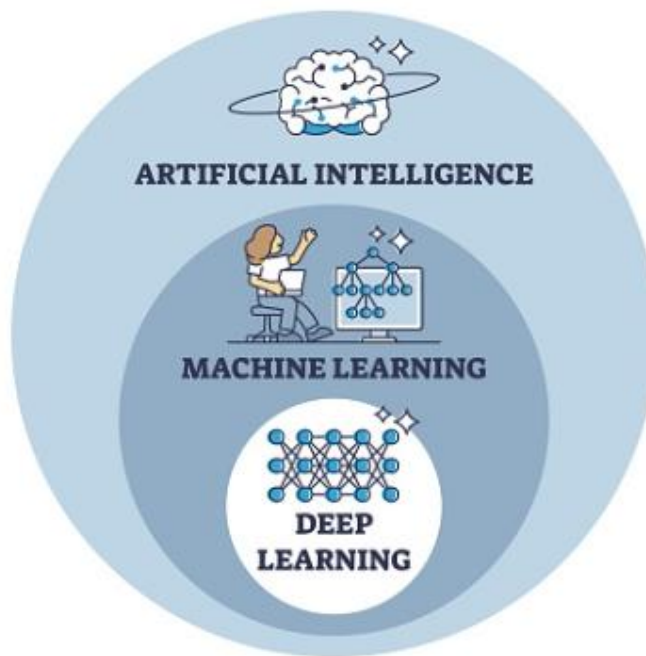
- 5.1 Diagramme de cas d'utilisation
 - 5.2 Diagramme de séquence (flux KYC complet)
 - 3.3.3 Diagramme de classes / MLD
-

La mise en œuvre d'un système d'automatisation du processus KYC repose sur un socle théorique et technologique solide. Ce chapitre pose les bases conceptuelles nécessaires à la compréhension des choix effectués, depuis les fondements de l'intelligence artificielle et de la vision par ordinateur jusqu'aux technologies retenues. Une fois ce cadre établi, il devient possible de traduire ces acquis en exigences concrètes, de concevoir une architecture adaptée et d'en produire les modèles UML qui serviront de blueprint à l'implémentation.

I. Fondements Théoriques

1. Intelligence Artificielle, Machine Learning et Deep Learning

Avant d'aborder la vision par ordinateur, il convient de présenter la hiérarchie conceptuelle sur laquelle elle repose. Ces trois disciplines s'emboîtent selon une relation de spécialisation croissante, chacune constituant un sous-ensemble plus restreint de la précédente.



[Figure 2.1 — Relation hiérarchique entre IA, ML et Deep Learning]

- **Intelligence Artificielle (IA)** : domaine regroupant l'ensemble des techniques permettant à une machine de simuler des comportements intelligents — apprentissage, raisonnement et prise de décision.
- **Machine Learning (ML)** : sous-discipline de l'IA dans laquelle les systèmes apprennent à partir de données, sans être explicitement programmés pour chaque tâche.
- **Deep Learning (DL)** : évolution du ML fondée sur des réseaux de neurones profonds, capables de traiter des données complexes (images, audio, texte). Le Deep Learning constitue aujourd'hui le moteur principal des systèmes de vision par ordinateur.

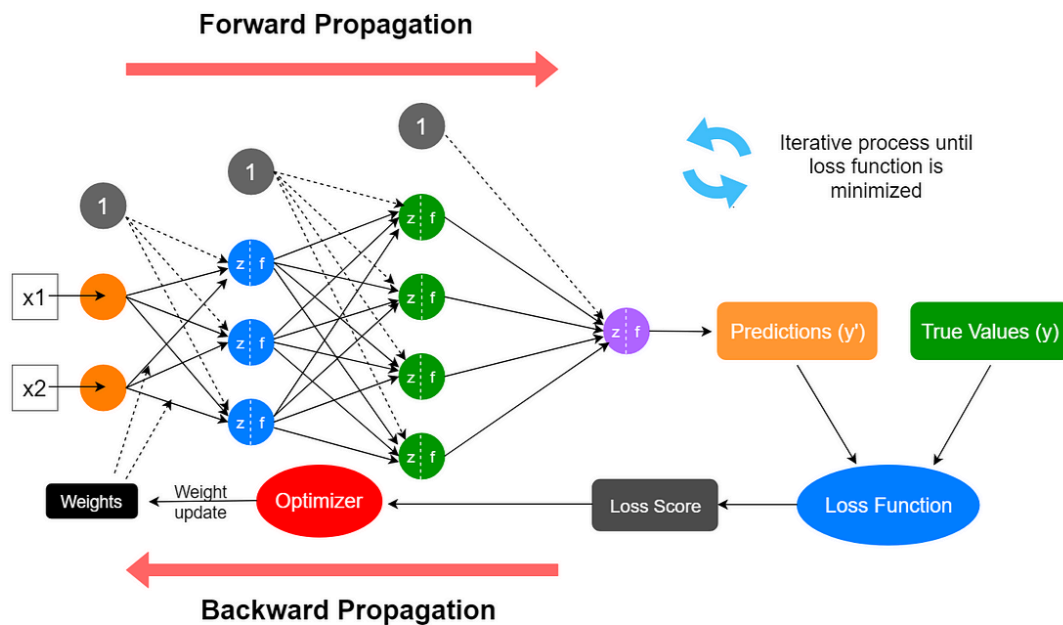
2. Vision par Ordinateur et Réseaux de Neurones Artificiels

La vision par ordinateur est une branche de l'intelligence artificielle qui permet aux machines d'interpréter et de comprendre le contenu visuel d'images ou de vidéos. Son objectif est de transformer des données visuelles brutes — des pixels — en informations sémantiques exploitables, reproduisant ainsi de façon automatisée les capacités du système visuel humain.

Dans le cadre du KYC, la vision par ordinateur intervient à travers plusieurs applications complémentaires et étroitement liées :

- **Reconnaissance d'images** : identification du type de document (carte nationale d'identité, passeport, ect).
- **Détection d'objets** : localisation des zones d'intérêt — photographie, champs textuels.
- **Reconnaissance optique de caractères (OCR)** : extraction et conversion du texte présent dans l'image en données numériques structurées.

Ces applications reposent sur les **réseaux de neurones artificiels**, brique fondamentale du Deep Learning. Inspirés du fonctionnement du cortex cérébral, ces réseaux sont composés de neurones interconnectés organisés en couches successives, chacune apprenant à extraire des représentations de plus en plus abstraites à partir des données d'entrée.



[Figure 2.2 — Propagation avant et rétropropagation dans un réseau de neurones]

L'architecture d'un réseau de neurones se décompose en trois types de couches :

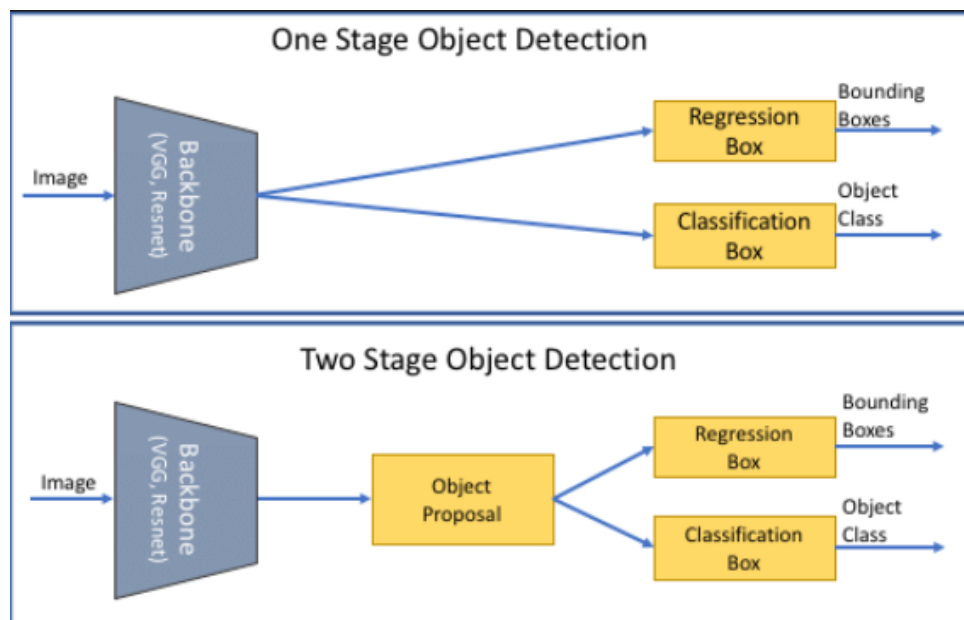
- **Couche d'entrée (Input Layer-Cercles orangés)** : reçoit les données brutes — pixels de l'image du document — et les transmet aux couches suivantes.
- **Couches cachées (Hidden Layers-Cercles bleus et verts)** : traitent l'information à travers plusieurs étapes de calcul afin d'extraire les caractéristiques complexes (contours, formes, zones de texte).

- **Classification d'image** : prédire la classe d'un objet unique dans l'image (ex. : « Ceci est une carte d'identité »).

- **Localisation d'objet** : identifier la présence d'un objet et indiquer sa position par une boîte englobante.
- **Détection d'objets** : identifier et localiser plusieurs objets et leurs classes respectives dans une même image.
- **Segmentation sémantique** : classer chaque pixel de l'image dans une catégorie, sans distinguer les différentes instances d'une même classe.
- **Segmentation d'instance** : identifier et délimiter chaque objet distinct — par exemple, séparer deux cartes d'identité disposées côte à côte.

3.2 Méthodes de Détection : One-Stage vs Two-Stage

Les méthodes de détection d'objets basées sur le Deep Learning se répartissent en deux grandes familles architecturales, dont le choix est guidé par le compromis entre vitesse d'inférence et précision.



[Figure 2.4 — Comparaison architecturale : détection One-Stage vs Two-Stage]

Méthodes One-Stage

Ces méthodes utilisent un réseau unique qui mappe directement l'image d'entrée vers les prédictions (boîtes englobantes et classes) en un seul passage. Leur architecture simplifiée leur confère une **vitesse d'inférence élevée**, idéale pour les applications en temps réel (vidéo, mobile). Exemples représentatifs : **YOLO, SSD**.

Méthodes Two-Stage

Elles procèdent en deux étapes distinctes : une première génère des propositions de régions d'intérêt (Region Proposals), et une seconde affine et classeifie ces propositions. Leur **précision est généralement supérieure**, au prix d'une latence plus élevée et d'un coût computationnel important. Exemples : **Faster R-CNN, R-CNN**.

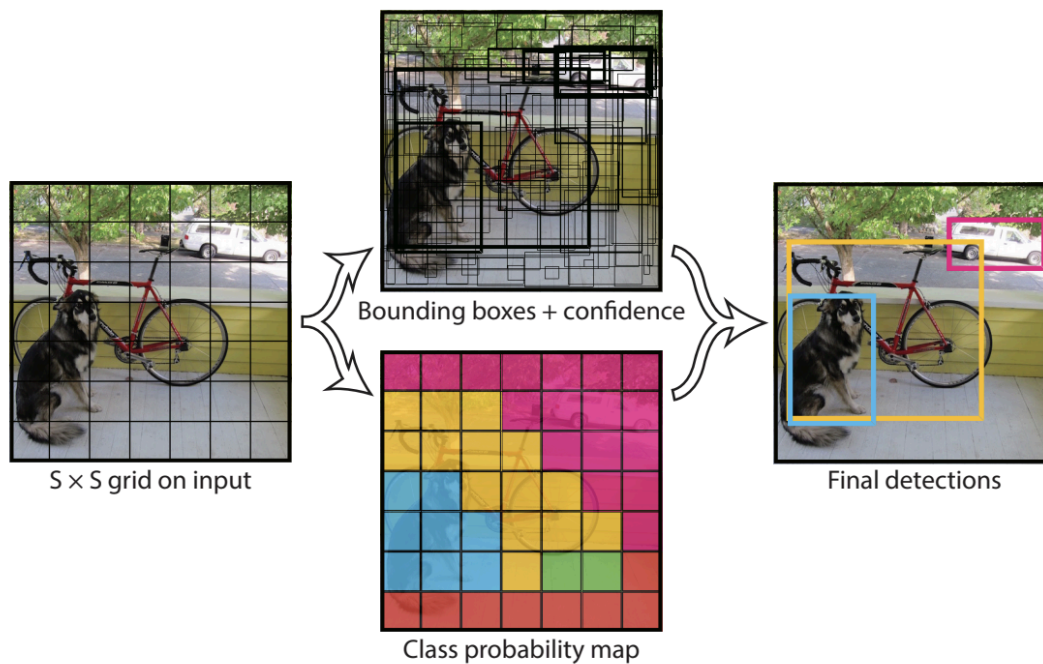
3.3 YOLO (You Only Look Once)



[Figure — Logo YOLO]

YOLO (You Only Look Once) est un algorithme de détection d'objets appartenant à la famille One-Stage, introduit en 2016 par Joseph Redmon et ses collaborateurs. Ce modèle a révolutionné la détection d'objets en proposant une approche unifiée, capable de détecter et classifier des objets en un seul passage réseau, atteignant des performances en temps réel tout en conservant une précision compétitive.

Principe de fonctionnement



[Figure 2.5b — Principe de fonctionnement de YOLO : grille $S \times S$, boîtes englobantes et carte de probabilités]

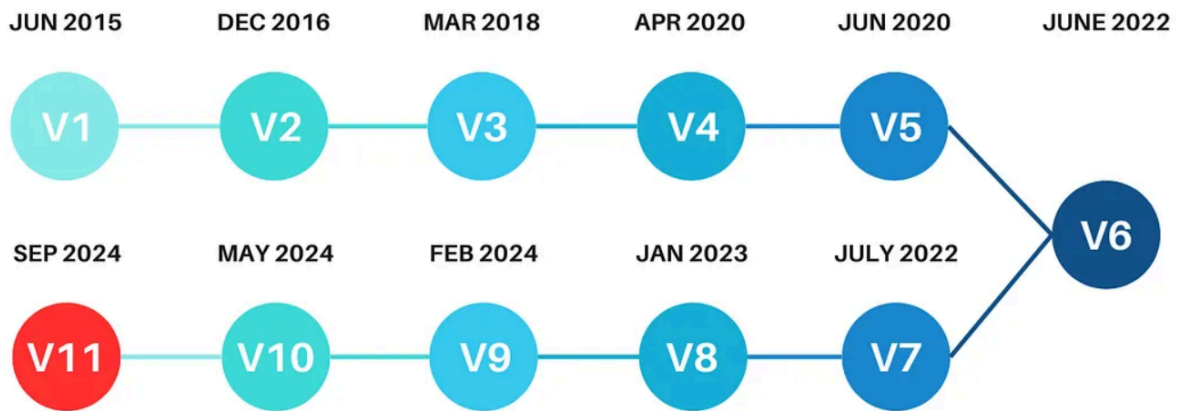
Comme illustré ci-dessus, YOLO adopte une approche globale et unifiée :

- L'image est divisée en une grille de cellules $S \times S$.
- Chaque cellule est responsable de la détection des objets dont le centre se situe dans sa zone.
- Pour chaque cellule, le modèle prédit simultanément : des **boîtes englobantes**, un **score de confiance** et la **classe de l'objet**.
- L'ensemble de ces prédictions est réalisé en un seul passage à travers le réseau, garantissant rapidité et efficacité.

Évolution de l'architecture YOLO

Depuis l'introduction de YOLOv1, le modèle a connu de nombreuses itérations, chacune améliorant la précédente en termes de précision, de rapidité et d'efficacité.

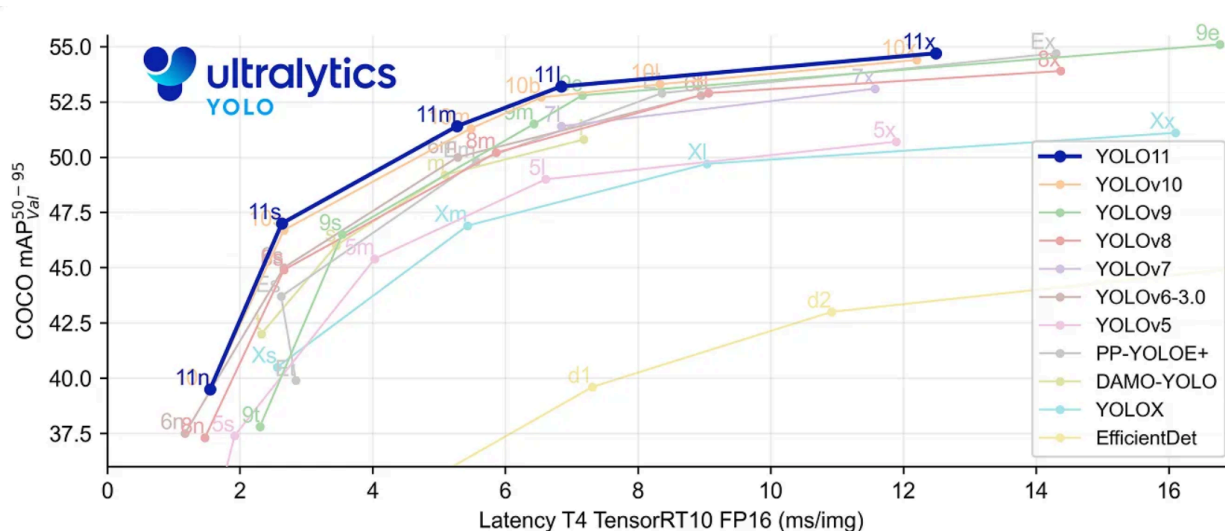
YOLO EVOLUTION TIME LINE



[Figure 2.6 — Chronologie de l'évolution des architectures YOLO (V1 à V11)]

- **YOLOv1 (2016)** : modèle original — performances temps réel, mais détection des petits objets limitée.
- **YOLOv2 (2017)** : introduction des boîtes d'ancrage, normalisation par lots et résolution accrue.
- **YOLOv3 (2018)** : prédictions multi-échelles via pyramides de features, améliorant la détection à différentes tailles d'objets.
- **YOLOv4 (2020)** : augmentation mosaïque et optimisation du backbone pour une inférence plus rapide.
- **YOLOv5 (2020)** : implémentation PyTorch largement adoptée, optimisée pour le déploiement pratique.
- **YOLOv6/v7 (2022)** : amélioration de la mise à l'échelle et de la précision, avec des versions compactes pour edge devices.
- **YOLOv8 (2023)** : backbone CSPDarkNet et agrégation de chemins améliorée, rehaussant vitesse et précision.
- **YOLO11 (2024)** : architecture C3K2 + SPPF + mécanisme d'attention C2PSA — meilleure détection des petits objets et efficacité computationnelle supérieure.

Analyse comparative sur COCO



[Figure 2.7 — Comparaison mAP50-95 / latence des modèles YOLO sur le dataset COCO]

La figure ci-dessus compare les modèles YOLO à l'aide de la métrique **mAP50-95** (mean Average Precision) sur le dataset COCO, en fonction de leur latence d'inférence. Les versions récentes offrent de meilleures performances, avec une précision accrue et une latence réduite. YOLO11 se distingue notamment par son efficacité : plus léger que ses homologues à haute précision, il conserve un excellent niveau de mAP, le rendant particulièrement adapté aux applications en temps réel.

Source : Ultralytics YOLO11 Documentation — <https://docs.ultralytics.com/fr/models/yolo11/>

Source : Terven et al. (2023) — A Comprehensive Review of YOLO Architectures in Computer Vision — <https://arxiv.org/abs/2304.00501>

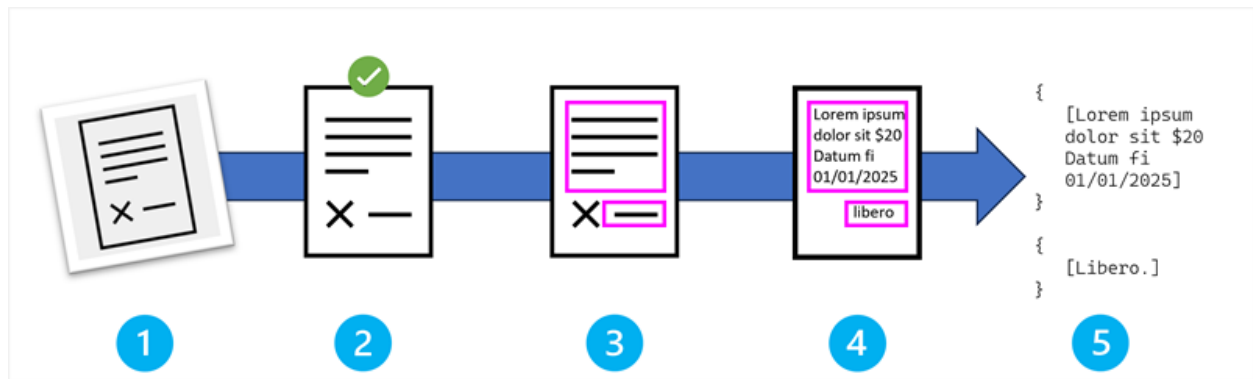
4. Reconnaissance Optique de Caractères (OCR)

La reconnaissance optique de caractères (**OCR — Optical Character Recognition**) est une technologie permettant d'extraire automatiquement du texte à partir d'images ou de documents numérisés. Elle transforme des informations textuelles visuelles en données numériques exploitables, facilitant leur traitement, leur stockage et leur analyse automatisée.

Dans le cadre des systèmes KYC, l'OCR joue un rôle **central** en permettant d'extraire les informations présentes sur les documents d'identité : nom, prénom, date de naissance, numéro d'identification, etc.

4.1 Pipeline OCR : Processus Pas à Pas

Le pipeline OCR se compose de cinq étapes séquentielles qui transforment une image brute en texte structuré exploitable :



[Figure 2.8 — Pipeline OCR en cinq étapes]

- **Étape 1 — Acquisition et entrée d'image :** le pipeline démarre à la réception de l'image. La qualité de celle-ci conditionne directement la précision finale de l'extraction de texte.
- **Étape 2 — Prétraitement et amélioration :** nettoyage et optimisation de l'image pour améliorer la lisibilité.
- **Étape 3 — Détection des régions textuelles :** localisation des zones contenant du texte au sein de l'image.
- **Étape 4 — Reconnaissance des caractères :** identification des caractères présents dans les zones détectées.
- **Étape 5 — Post-traitement et sortie :** structuration et correction des données extraites.

Ce processus permet de transformer une image contenant du texte en un format exploitable par un système informatique.

4.2 Benchmark des Moteurs OCR Open Source

Plusieurs solutions OCR open source ont été développées, présentant des performances variables en termes de précision, de robustesse et de vitesse. Les approches basées sur le Deep Learning, telles que PaddleOCR, se révèlent généralement plus performantes en conditions réelles (flou, bruit, faible éclairage), ce qui les rend particulièrement adaptées aux systèmes KYC.

Moteur	Architecture	Précision imprimé	CER moyen	Vitesse CPU	Arabe	Fine-tuning	Licence
Tesseract	LSTM + HMM	~90 %	3–5 %	Très rapide	Oui	Oui	Apache 2.0
PaddleOCR	CNN + Transformer	~93 %	2–4 %	Moyen	Oui	Oui	Apache 2.0
EasyOCR	CRAFT + CRNN	~85 %	5–8 %	Lent	Oui	Partiel	Apache 2.0
Kraken v5	BLLA + CLSTM	~82 %	2–5 %	Rapide	Excellent RTL	Oui	Apache 2.0
MMOCR v1	CNN / Transformer	~84 %	4–7 %	Moyen	Partiel	Oui	Apache 2.0
TrOCR	ViT + BEiT	~91 %	2–4 %	Lent (GPU)	Non off.	Oui	MIT

Tableau 2.1 — Benchmark des principaux moteurs OCR open source (mise à jour 2025)

Les scores de précision sont des moyennes indicatives sur texte imprimé propre (300 DPI+). Le CER (Character Error Rate) varie significativement selon la qualité du scan, la langue et le domaine. PP-OCRv5 est à ce jour le seul moteur à combiner support natif de l'arabe (RTL), fine-tuning aisé et analyse de mise en page avancée.

Source : IntuitionLabs — Technical Analysis of Modern Non-LLM OCR Engines (fév. 2026) — intuitionlabs.ai

Source : arXiv 2510.03570 — Evaluating OCR performance: Tesseract, EasyOCR, PaddleOCR, TrOCR benchmark (2025) — arxiv.org/abs/2510.03570

II. Choix Technologiques

Les fondements théoriques présentés précédemment guident naturellement vers la sélection des outils les plus adaptés au contexte du projet. Deux technologies ont été retenues après une analyse comparative rigoureuse : YOLO11m pour la détection d'objets et PP-OCRv5 pour la reconnaissance de caractères.

1. Modèle de Détection : YOLO11m

1.1 Présentation de YOLO11

YOLO11 est la dernière génération de modèles de détection d'objets en temps réel développée par Ultralytics, publiée en septembre 2024. Il introduit plusieurs innovations architecturales majeures par rapport à ses prédécesseurs :

- **Bloc C3K2** (Cross Stage Partial with Kernel Size 2) : extraction de caractéristiques spatiales plus riche.
- **Module SPPF** (Spatial Pyramid Pooling Fast) : meilleure capture multi-échelle des objets de différentes tailles.
- **Bloc C2PSA** (Convolutional with Parallel Spatial Attention) : amélioration de la détection des petits objets.

La famille YOLO11 propose cinq variantes (n, s, m, l, x) couvrant un spectre allant des dispositifs embarqués aux serveurs GPU haute performance.

1.2 Comparaison des Variantes YOLO11

Le tableau suivant présente les performances benchmarkées sur le dataset COCO (mAP50-95) pour chaque variante, en précisant le nombre de paramètres et la vitesse d'inférence CPU.

Variante	Paramètres	GFLOPs	mAP ⁵⁰⁻⁹⁵ (%)	Vitesse CPU (ms)	Usage recommandé
YOLO11n	2,6 M	6,5	39,5	56,1	Edge / mobile
YOLO11s	9,4 M	21,5	47,0	90,0	Léger temps réel
YOLO11m ✓	20,1 M	68,0	51,5	183,2	Équilibre optimal
YOLO11l	25,3 M	86,9	53,4	232,6	Haute précision

Variante	Paramètres	GFLOPs	mAP ⁵⁰⁻⁹⁵ (%)	Vitesse CPU (ms)	Usage recommandé
YOLO11x	56,9 M	194,9	54,7	462,8	Précision maximale

Tableau 2.2 — Comparaison des variantes YOLO11 sur COCO val2017 (Source : Documentation officielle Ultralytics)

La variante **YOLO11m** offre le meilleur équilibre entre précision et complexité computationnelle pour notre contexte d'application. Les variantes légères (n, s) privilégient la vitesse au détriment de la précision, tandis que les modèles lourds (l, x) engendrent des coûts incompatibles avec les contraintes d'un système bancaire opérationnel.

1.3 Justification du Choix : YOLO11m vs YOLOv8m

Le tableau ci-dessous compare YOLO11m à YOLOv8m, qui constitue la génération précédente la plus répandue en production.

Critère	YOLO11m (retenu)	YOLOv8m	Avantage
Paramètres	20,1 M	25,9 M	-22 % (YOLO11m)
mAP50-95 (COCO)	51,5 %	50,2 %	+1,3 pt (YOLO11m)
Vitesse CPU (ms)	183,2	~220	Plus rapide
Architecture	C3K2 + SPPF + C2PSA	C2f + SPPF	Extraction enrichie
Déploiement	Edge / Cloud / GPU	Edge / Cloud / GPU	Équivalent

Tableau 2.3 — YOLO11m vs YOLOv8m : gains en précision et efficacité (Source : Ultralytics)

YOLO11m a été retenu car il offre le meilleur compromis entre précision (mAP 51,5 %) et coût computationnel, tout en nécessitant 22 % de paramètres en moins que YOLOv8m. Cette légèreté le rend plus adapté à une intégration dans un système bancaire contraint en ressources matérielles.

2. Moteur OCR : PaddleOCR PP-OCRv5

2.1 Présentation de PP-OCRv5

PP-OCRv5 est la cinquième génération du pipeline OCR de PaddlePaddle (Baidu), publiée dans le cadre de PaddleOCR 3.0 en 2024. Il représente une évolution majeure conçue spécifiquement pour la reconnaissance de texte dans des environnements variés et multilingues, incluant des documents réels, du texte manuscrit et des images capturées en conditions non contrôlées.

L'architecture repose sur un pipeline à deux étages : **détection des zones de texte** (PP-OCRv5_det), puis **reconnaissance caractère par caractère** (PP-OCRv5_rec), disponible en variante mobile (optimisée CPU) et serveur (optimisée GPU).

2.2 Performances et Évolution par Rapport aux Versions Précédentes

Critère	PP-OCRv3	PP-OCRv4	PP-OCRv5 (retenu)
Langues supportées	80+	80+	106 (arabe inclus)
Amélioration précision	Référence	+10–15 %	+30 % multilangue / +13 pts global
Support arabe	Partiel	Partiel	Natif (2 676 images d'entraînement)
Texte déformé / incliné	Limité	Moyen	Amélioré (manuscrit, vertical)
Variantes disponibles	Mobile seul	Mobile + Server	Mobile + Server
Performance vs VLM	—	—	Surpasse GPT-4o, Gemini 2.5 Pro

Tableau 2.4 — Évolution des performances PP-OCR à travers les versions (Sources : PaddleOCR Documentation & PaddleOCR 3.0 Technical Report)

Source : [Documentation PP-OCRv5 — paddlepaddle.github.io](https://paddlepaddle.github.io)

2.3 Critères de Sélection pour le Contexte Tunisien

Le choix de PP-OCRv5 s'appuie sur quatre critères directement liés aux contraintes du projet KYC de la BTE.

a) Support natif de l'arabe

PP-OCRv5 couvre 106 langues, dont l'arabe, le français et l'anglais, avec un dataset arabe dédié de 2 676 images d'entraînement. L'arabe étant une langue à écriture cursive et de droite à gauche (RTL), la majorité des solutions OCR classiques échouent sur ce type de texte. PP-OCRv5 intègre nativement cette complexité, le rendant parfaitement adapté aux documents d'identité tunisiens bilingues (arabe / français).

b) Amélioration de précision documentée

Comparé à PP-OCRv3, PP-OCRv5 atteint une amélioration de plus de 30 % en précision pour la reconnaissance multilingue, et +13 points de pourcentage d'amélioration globale par rapport à PP-OCRv4. Ces gains sont particulièrement significatifs sur les scénarios de texte déformé, incliné ou manuscrit — conditions précisément rencontrées lors de la capture mobile d'images KYC.

c) Performance face aux grands modèles de vision (VLM)

Dans les benchmarks publiés sur OmniDocBench, PP-OCRv5 surpasse des modèles de grande taille tels que GPT-4o, Gemini 2.5 Pro et Qwen2.5-VL sur les tâches OCR spécialisées, tout en utilisant moins de 100 millions de paramètres. Cette efficacité est essentielle pour un déploiement on-premise dans un environnement bancaire aux ressources définies.

d) Licence open source et facilité d'intégration

PP-OCRv5 est distribué sous licence **Apache 2.0**, sans restriction commerciale. Il dispose d'une API Python claire et d'une intégration native avec PaddlePaddle 3.0, permettant une incorporation directe dans le pipeline KYC de la banque, sans dépendance à des services cloud tiers.

PP-OCRv5 représente à ce jour l'une des rares solutions open source capables de traiter de manière robuste des documents bilingues arabe/français dans des conditions réelles de capture mobile — ce qui constitue précisément le défi central du projet KYC de la BTE.

III. Spécification des Besoins

Cette section sera complétée ultérieurement avec les besoins fonctionnels et non fonctionnels du système.

1. Besoins Fonctionnels

[À compléter]

2. Besoins Non Fonctionnels

[À compléter]

IV. Architecture Globale de la Solution

1. Architecture Logique

[Frontend / Backend / IA / DB — À compléter]

2. Architecture Physique

[On-premise, flux de données — À compléter]

V. Conception UML

1. Diagramme de Cas d'Utilisation

[À compléter]

2. Diagramme de Séquence

[Flux KYC complet — À compléter]

3. Diagramme de Classes et MLD

[À compléter]

Conclusion du chapitre — Ce chapitre a établi les fondements théoriques et technologiques du système. L'ancrage dans les principes de l'IA, du Deep Learning et de la vision par ordinateur a permis de justifier rigoureusement le choix de YOLO11m pour la détection d'objets et de PP-OCrv5 pour la reconnaissance de texte arabe et français. Ces deux briques technologiques, couplées à l'architecture et à la conception UML détaillées, forment le socle sur lequel s'appuie l'implémentation décrite dans le chapitre suivant.